

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I **M. Phanith Chandu (192425238)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Phanith Chandu

Q. No	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Score (100%)
1						
2						
3						
4						
5						
Total Marks (Out of 100)					Total Marks (Out of 50)	

Annexure A

Pseudocode Writing Task (Algorithm Design)

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline

A system must process a live video stream to:

- A. Capture frames from a camera
- B. Detect faces in each frame
- C. Recognize faces using a pre-trained dataset
- D. Display bounding boxes and labels in real time

Constraints:

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
- B. Enhance image quality (noise removal, contrast improvement)
- C. Segment text regions
- D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

3. Motion Detection and Tracking System

A surveillance system must:

- A. Read video input
- B. Detect motion between consecutive frames
- C. Track moving objects
- D. Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

4. Vehicle Counting System using Video

A traffic monitoring system must:

- A. Read video frames
- B. Detect vehicles entering a region of interest (ROI)
- C. Track movement across frames
- D. Count vehicles and display count

Constraints:

- Avoid double counting
- Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.

5. Video Frame Extraction System

A system must:

- A. Read video input
- B. Extract frames at fixed intervals
- C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

1. Real-Time Face Detection and Recognition Pipeline

Problem Understanding

- Input: live camera/video stream.
- Stages: capture → preprocess → detect faces → recognize faces → display.
- Output: live video with bounding boxes and labels.
- Constraints: continuous operation, low delay, and multiple faces.

Pseudocode

BEGIN

 OPEN camera

 LOAD face detection model

 LOAD face recognition model and known-face dataset

 IF camera is not available THEN DISPLAY

 "Camera Error"

 STOP

 END IF

 WHILE camera is open READ

 frame

 IF frame is empty THEN BREAK

 RESIZE frame if needed

 CONVERT frame to required color format

 PREPROCESS frame

 DETECT all faces

 FOR each detected face EXTRACT face

 region PREPROCESS face region

 COMPARE face with known dataset

 IF match score $\geq$ recognition threshold THEN label $\leftarrow$ matched name

 ELSE

 label $\leftarrow$ "Unknown" END IF

 DRAW bounding box

 DRAW label

 END FOR

 DISPLAY frame

 IF exit key is pressed THEN BREAK END

WHILE

RELEASE camera CLOSE

windows

END

2. OCR Pipeline for Document Processing

Problem Understanding

- Input: batch of printed-text document images.
- Stages: read → remove noise → improve contrast → segment text → recognize text.
- Output: recognized text displayed and stored.
- Constraints: noisy/low-contrast images and batch processing.

Pseudocode

BEGIN

 GET list of input document images

 FOR each imageFile READ

 image

 IF image cannot be read THEN DISPLAY error

 CONTINUE

 END IF

 CONVERT image to grayscale REMOVE

 noise using blur/filter IMPROVE contrast

 APPLY thresholding

 APPLY morphological cleaning SEGMENT text

 regions

 FOR each text region EXTRACT

 region

 SEND region to OCR module STORE

 recognized text

 END FOR

 DISPLAY recognized text SAVE

 recognized text

 END FOR

 DISPLAY "Batch OCR Completed"

END

3. Motion Detection and Tracking System

Problem Understanding

- Input: continuous surveillance video.
- Stages: read frames → frame difference → filtering → motion regions → tracking → visualization.
- Output: highlighted moving objects.
- Constraints: dynamic background and near-real-time processing.

Pseudocode

```
BEGIN
  OPEN video source READ
  first frame
  CONVERT first frame to grayscale SET previousFrame ← first frame INITIALIZE tracker

  WHILE video is available READ current
    frame
    IF frame is empty THEN BREAK

    CONVERT current frame to grayscale APPLY Gaussian
    blur

    difference ← ABSOLUTE_DIFFERENCE(currentFrame, previousFrame) APPLY threshold to
    difference
    APPLY morphological filtering FIND motion

    contours/regions FOR each motion region

      IF area > minimumArea THEN
        CREATE bounding box ADD
        region to tracker
      ENDIF
    END FOR

    UPDATE tracker
    DRAW tracked bounding boxes
    DISPLAY frame

    previousFrame ← currentFrame

    IF exit key is pressed THEN BREAK END
  WHILE
RELEASE video CLOSE
windows
END
```

4. Vehicle Counting System using Video

Problem Understanding

- Input: traffic video.
- Stages: detect vehicles → track IDs → check ROI/counting line → count → display.
- Output: total vehicle count.
- Constraints: avoid double counting and handle multiple vehicles.

Pseudocode

```
BEGIN
    OPEN traffic video DEFINE
    ROI
    DEFINE countingLine INITIALIZE
    vehicle detector INITIALIZE tracker
    vehicleCount ← 0
    countedIDs ← EMPTY_SET

    WHILE video is available READ frame
        IF frame is empty THEN BREAK DETECT

        vehicles in/near ROI

        FOR each detected vehicle
            ASSIGN or UPDATE unique vehicleID UPDATE
            vehicle position
            DRAW bounding box and ID

            IF vehicle crosses countingLine THEN
                IF vehicleID NOT IN countedIDs THEN
                    vehicleCount ← vehicleCount + 1 ADD
                    vehicleID to countedIDs
                ENDIF
            END IF
        END FOR

        DRAW ROI and countingLine
        DISPLAY vehicleCount

        DISPLAY frame

        IF exit key is pressed THEN BREAK END
    WHILE
        RELEASE video
        CLOSE windows
    END
```

5. Video Frame Extraction System

Problem Understanding

- Input: video file.
- Stages: read frames → check fixed interval → save selected frame.
- Output: selected frames stored in an output folder.
- Constraints: efficient storage and no redundant frames.

Pseudocode

```
BEGIN
  OPEN video
  GET total frames / frame
  rate SET interval
  frameIndex ← 0
  savedCount ← 0

  WHILE video has
    frames
    READ frame
    IF frame is empty THEN BREAK

    IF frameIndex MOD interval = 0
      THEN CREATE unique
        output filename SAVE frame
        savedCount ←
        savedCount + 1 END IF

    frameIndex ←
    frameIndex + 1 END
  WHILE
    RELEASE video
    DISPLAY savedCount

  END
```

Key Points:

- MOD selects frames at fixed intervals.
- Only selected frames are stored, reducing storage use.
- frameIndex tracks the current frame.
- savedCount creates sequential output names.
- The loop ends when the video has no more frames.
- For example, interval = 30 saves every 30th frame.